OWNSTUDIO · LAB · TEST SEQUENCE EDITOR · MOCK-UP

Test sequence editor

The reference-step procedure from the buck example, built with Blockly in a flat OwnStudio theme. Toolbox on the left, sequence in the middle, the generated Python on the right. The "Step response" block is a user-defined block: it is defined once (bottom of the workspace) and used as one line in the sequence.

BLOCKS

- Board
- Power
- Instruments
- Capture · fast
- Monitor · slow
- Flow
- Checks
- My blocks

VARIABLES

`wVoltageRef_V` `Mode` `V_LOW1` `I_LOW1` `V_HIGH` `source.voltage` `load.ohm`

From the board's data objects, the source and load nodes, and the probes on the canvas. Drag one into a slot.

Test sequence `BUCK-REF-01` on `board` — START

check setup: board `Twist 1.4.2` runs `this setup's firmware`

ask the user: load on LOW1 = `load.ohm`

set `wVoltageRef_V` to `10 V`

switch the converter on — ASKS TO CONFIRM

repeat for `step` in `[10→12 V, 12→10 V]`

- ▸ Step response `step` capture `V_LOW1` `I_LOW1` — MY BLOCK
- if settling time ≤ `10 ms`
  - record `PASS` REQ-TRACKING
- else
  - record `FAIL` with the measured time

monitor `V_HIGH` `I_LOW1` every `100 ms` during `2 s`

switch the converter off

simulate the same sequence and overlay `drop a capture here`

DEFINITION OF "STEP RESPONSE" (EXPANDED)

- ▾ define Step response `step` capture `channels`
- arm the scope on `channels` for `100 ms` , `20 ms` before the trigger
- set `wVoltageRef_V` to `step.to` — TRIGGER
- read the capture into `waveform`
- settling time = time for `V_LOW1` to stay within `2 %` of `step.to`

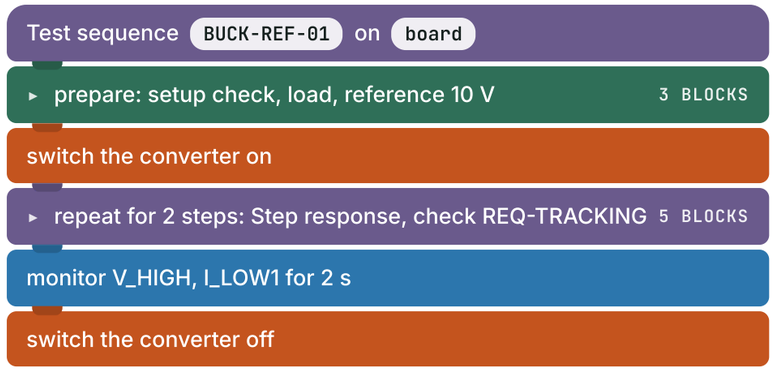
GENERATED PYTHON · READ ONLY

```
# BUCK-REF-01, generated by the editor.
# Hand edits become a "custom step" block.
def step_response(t, step, channels):
    t.arm(channels, window_s=0.100,
          pretrigger_s=0.020)
    t.update(wVoltageRef_V=step.to)
    waveform = t.capture()
    return waveform, settling_time(
        waveform["V_LOW1"], step.to, band=0.

def run(t, ctx):
    t.check_setup()
    load = ctx.ask("Load on LOW1 (ohm)")
    t.update(wVoltageRef_V=10.0)
    t.power_on()          # OwnStudio asks
    for step in STEPS:
        w, ts = step_response(t, step,
                              ["V_LOW1", "I_
        if ts ⩽ 0.010:
            ctx.record("REQ-TRACKING", PASS)
        else:
            ctx.record("REQ-TRACKING", FAIL,
    t.monitor(["V_HIGH", "I_LOW1"],
              period_s=0.1, duration_s=2)
    t.power_off()
    ctx.twin()
```

Colours follow the probe modes: amber = Capture (fast, scope) blue = Monitor (slow, ThingSet) orange = power, always confirmed

The same sequence, collapsed



Any block with children folds to one line showing how many blocks it hides. This is the answer to the Jenga tower.

What makes it readable

Nesting. C-shaped blocks (repeat, if/else, "while the converter is on") hold other blocks; the indentation is the structure.

Your own blocks. A group of blocks becomes a named block with inputs ("Step response"), used as one line, defined once.

Collapse. Any container folds to one line with a count.

Sentences, not code. Blocks read as instructions with slots; variables are dragged from the list built from the setup.

Structured only. No arbitrary jumps, so the Python is clean and the editor can read back its own output.

Theme. Blockly's flat renderer with OwnStudio's colours, no puzzle-piece cartoon: only a small notch shows where blocks join.